

# Systeme intelligent multi-modeles pour la retention client

Projet M1 Dev — EFREI 2025-26 — Sujet 2 Auteur : Omar Benfdila Date : mai 2026  
Dépôt du projet : <https://github.com/oumss91370/churn-retention-analytics>

---

## 1. Contexte et problématique

Les opérateurs d'abonnement perdent en moyenne 5 à 7 % de leur base client chaque mois. Le coût de remplacement d'un client perdu dépasse de cinq à vingt-cinq fois celui de sa rétention. Les services CRM disposent de signaux comportementaux riches (usage, paiement, support, satisfaction) mais sans outil pour les traduire en priorisation d'action.

Le projet répond à une décision opérationnelle précise : **identifier en avance les clients à risque de churn pour permettre une action ciblée**. Le modèle joue le rôle d'un système de scoring qui transforme un signal noyé dans 31 variables en une probabilité par client, accompagnée d'une explication. Le dataset comporte 10 000 clients dont 10,21 % sont des churners — un déséquilibre marqué qui conditionne tous les choix techniques qui suivent.

---

## 2. Compréhension des données et analyse exploratoire (EDA)

L'EDA suit une approche orientée modélisation : chaque résultat produit une décision concrète sur la suite du pipeline. Les techniques mobilisées sont les statistiques descriptives (`describe`, `value_counts`), l'analyse des distributions par classe via histogrammes superposés et boxplots, la détection d'outliers par la méthode interquartile (IQR), et l'étude des corrélations via la matrice de Pearson.

L'audit qualité révèle qu'une seule colonne porte des valeurs manquantes : `complaint_type`, à hauteur de 20,45 %. Ces vides n'expriment pas un défaut de saisie mais l'absence de plainte — une information à conserver. La distribution de la cible confirme le ratio 1:9 du déséquilibre. Cinq variables ressortent comme discriminantes : score NPS, score CSAT, ancienneté en mois (`tenure_months`), dernière connexion (`last_login_days_ago`) et nombre de tickets de support. Les churners affichent un NPS moyen de -10 contre +25 chez les fidèles, et un CSAT moyen de 2,9 contre 3,6.

La méthode IQR identifie moins de 2 % de valeurs extrêmes sur les variables clés. Ces points ne sont pas des erreurs : un client à cinq échecs de paiement est précisément le profil cible. Aucun outlier n'est supprimé. La matrice de corrélation montre que `total_revenue` est mécaniquement lié à `monthly_fee` × `tenure_months` (multicolinéarité connue), ce qui pousse à dériver une variable binaire `is_high_value` plutôt qu'à éliminer une colonne. Les corrélations Pearson avec `churn` plafonnent à 0,16 en valeur absolue : aucune variable seule n'explique le churn, le modèle devra combiner plusieurs signaux.

[image]

*Lecture : pour chaque variable, la classe churn (rouge) se décale visiblement de la classe fidèle*

(bleu) sur le NPS, le CSAT et la dernière connexion — ce sont ces séparations qui justifient leur conservation comme variables prédictives, tandis que des distributions superposées (ex. âge) annoncent un faible pouvoir discriminant.

---

### 3. Transformation des données et qualité des données

Quatre transformations majeures sont appliquées avant la modélisation, encapsulées dans un Pipeline scikit-learn fitté uniquement sur le train pour éviter toute fuite.

**Traitement des valeurs manquantes.** Les NaN de `complaint_type` sont remplacés par la chaîne "None", qui devient une catégorie à part entière.

**Encodage des variables catégorielles.** Un `OneHotEncoder(handle_unknown='ignore')` transforme les 14 colonnes catégorielles en colonnes binaires. L'option `handle_unknown` rend l'encodeur robuste aux nouvelles modalités apparaissant en production.

**Standardisation des variables numériques.** Un `StandardScaler` centre-réduit les 18 variables continues. Cette étape est cruciale pour les modèles sensibles à l'échelle (régression logistique, MLP) car les ordres de grandeur vont de 1-10 (CSAT) à plusieurs milliers (revenu total).

**Création de quatre variables dérivées :**

- `tickets_per_month = support_tickets / (tenure_months + 1)` — normalise le volume de tickets par l'ancienneté
- `failed_payment_rate = payment_failures / (tenure_months + 1)` — taux d'échec de paiement
- `is_high_value = 1` si `total_revenue > médiane train`, 0 sinon — drapeau client à forte valeur
- `engagement_drop = 1` si `usage_growth_rate < 0`, 0 sinon — drapeau désengagement

Le split train/test est stratifié 80/20 (`random_state=42`) pour préserver le ratio 10,21 % de churn dans les deux ensembles. L'absence de fuite est garantie par un test automatique (CRIT-01) qui vérifie que les statistiques du scaler correspondent exactement aux données du train.

---

### 4. Modélisation et choix des algorithmes

Quatre modèles sont entraînés et comparés, choisis pour couvrir les familles d'algorithmes pertinentes pour ce problème.

**Régression logistique** sert de baseline interprétable. C'est un modèle linéaire généralisé qui modélise la probabilité de churn via la fonction sigmoïde. Sa frontière de décision est une droite dans l'espace standardisé — utile pour mesurer ce qu'apporte la complexité.

**Random Forest** est un ensemble d'arbres de décision capable de capter des interactions non linéaires entre variables. Il est robuste à la multicollinéarité (importante ici à cause du couple `total_revenue / monthly_fee × tenure_months`) et fournit nativement des importances de variables.

**XGBoost** est l'état de l'art en gradient boosting sur données tabulaires. Sa régularisation forte et son optimisation séquentielle des arbres en font un candidat naturel pour ce type de problème.

**MLP (Multi-Layer Perceptron)** joue le rôle de témoin Deep Learning. Architecture dense 64 → 32 avec activation ReLU, optimiseur Adam et early stopping sur la loss de validation. L'implémentation utilise `MLPClassifier` de scikit-learn faute de compatibilité TensorFlow 2.16 avec Python 3.13. L'enjeu pédagogique est de vérifier si un réseau neuronal apporte un gain sur ce dataset tabulaire de 10 000 lignes ou si un modèle d'arbres suffit.

La gestion du déséquilibre fait l'objet d'une comparaison empirique systématique. Trois techniques sont confrontées sur la régression logistique avec seuil tuné : `class_weight='balanced'` (pondération côté loss), SMOTE (oversampling synthétique) et Random Under-Sampling (sous-échantillonnage de la classe majoritaire). SMOTE et Random Under-Sampling sont encapsulés dans un `imblearn.Pipeline` pour garantir leur application uniquement sur le train.

| Technique                            | F1 sur test  |
|--------------------------------------|--------------|
| Baseline (sans rééquilibrage)        | 0,316        |
| <code>class_weight='balanced'</code> | <b>0,323</b> |
| SMOTE                                | 0,310        |
| Random Under-Sampling                | 0,305        |

`class_weight='balanced'` est retenu : il offre le meilleur F1 sans modifier la distribution des données. SMOTE n'apporte rien sur ce dataset, probablement parce que les churners n'ont pas une structure homogène qui se prête à l'interpolation.

---

## 5. Évaluation des modèles

Les métriques choisies répondent à la nature déséquilibrée du problème : **F1**, **Recall**, **Precision**, **PR-AUC** (Precision-Recall Area Under Curve) et **ROC-AUC**. L'accuracy seule est volontairement écartée car elle dépasserait 0,90 même pour un modèle qui ne détecterait aucun churning.

La validation s'appuie sur une **cross-validation stratifiée 5-fold** sur le train, ce qui garantit que chaque fold conserve la proportion de churn. Les hyperparamètres de Random Forest et XGBoost sont optimisés par `RandomizedSearchCV` (15 itérations, scoring `average_precision`, 3 folds). Le **seuil de décision** est ensuite ajusté entre 0,05 et 0,95 par pas de 0,05 pour maximiser le F1, étape critique en déséquilibre.

| Modèle                | F1           | Recall       | Precision | PR-AUC | ROC-AUC | Temps |
|-----------------------|--------------|--------------|-----------|--------|---------|-------|
| Régression logistique | 0,323        | 0,343        | 0,304     | 0,243  | 0,729   | < 1 s |
| <b>Random Forest</b>  | <b>0,407</b> | <b>0,789</b> | 0,274     | 0,303  | 0,807   | ~30 s |
| XGBoost               | 0,384        | 0,760        | 0,257     | 0,315  | 0,805   | ~6 s  |
| MLP                   | 0,287        | 0,348        | 0,244     | 0,235  | 0,705   | < 1 s |

Le **Random Forest** est retenu : meilleur F1 et meilleur Recall, deuxième PR-AUC, temps d'entraînement maîtrisé, interprétabilité native via `feature_importances_` et compatibilité directe avec SHAP `TreeExplainer`. Le MLP ne montre aucun avantage sur ce dataset tabulaire — confirmation pédagogique que le Deep Learning n'est pas

universellement supérieur. Sur le hold-out 20 % (2 000 clients dont 204 churners réels), le modèle final atteint  $F1 = 0,407$ ,  $Recall = 0,789$ ,  $ROC-AUC = 0,807$ , avec un seuil opérationnel de 0,35 et une matrice de confusion ( $TN = 1\,369$ ,  $FP = 427$ ,  $FN = 43$ ,  $TP = 161$ ). La précision modérée est un compromis assumé : dans un contexte CRM, le coût d'une action de rétention est faible devant le coût d'un client perdu.

[image]

*Lecture : la courbe reste nettement au-dessus de la ligne de base (10,2 % de churners), ce qui confirme un vrai pouvoir de classement ; sa décroissance rapide quand le rappel augmente illustre le compromis assumé — capter plus de churners se paie en précision, d'où le seuil retenu à 0,35.*

---

## 6. Interprétation et analyse des résultats

L'interprétabilité repose sur deux techniques complémentaires. Les **importances natives** du Random Forest hiérarchisent les variables selon leur contribution à la réduction de l'impureté (indice de Gini) — une vue globale du comportement du modèle. **SHAP** avec **TreeExplainer** apporte ensuite une explication à la fois globale (moyenne des contributions sur 300 instances de test) et locale (par client individuel), ce qui confirme et affine le classement des importances natives.

L'agrégation business des valeurs SHAP, qui consolide les modalités one-hot sous leur variable d'origine, fait émerger trois leviers actionnables :

1. **Score CSAT** (impact moyen 0,07). Un CSAT inférieur à 2 est un précurseur direct du churn dans les 30 jours. *Recommandation* : remonter chaque  $CSAT \leq 2$  au manager pour un rappel client dans les 48 heures.
2. **Ancienneté** (impact moyen 0,05). Les nouveaux clients ( $< 6$  mois) constituent une zone sensible. *Recommandation* : onboarding renforcé sur les 90 premiers jours avec mesure NPS à J30, J60, J90.
3. **Connexions mensuelles** (impact moyen 0,035). En dessous de cinq connexions mensuelles, le risque double. *Recommandation* : campagne de réactivation produit ciblée sur les fonctionnalités sous-utilisées.

Les force plots locaux sur trois clients-types (haut risque, bas risque, cas limite) montrent que le modèle combine deux à quatre signaux par décision. Pour un client à risque limite (probabilité 0,35), les facteurs déclencheurs sont les échecs de paiement, ce qui suggère qu'un rappel ciblé de relance de paiement pourrait inverser la trajectoire.

[image]

*Lecture : la longueur de chaque barre mesure l'impact moyen de la variable sur la prédiction. Le CSAT, l'ancienneté et les connexions mensuelles dominent — ce trio guide directement les actions de rétention recommandées ci-dessus.*

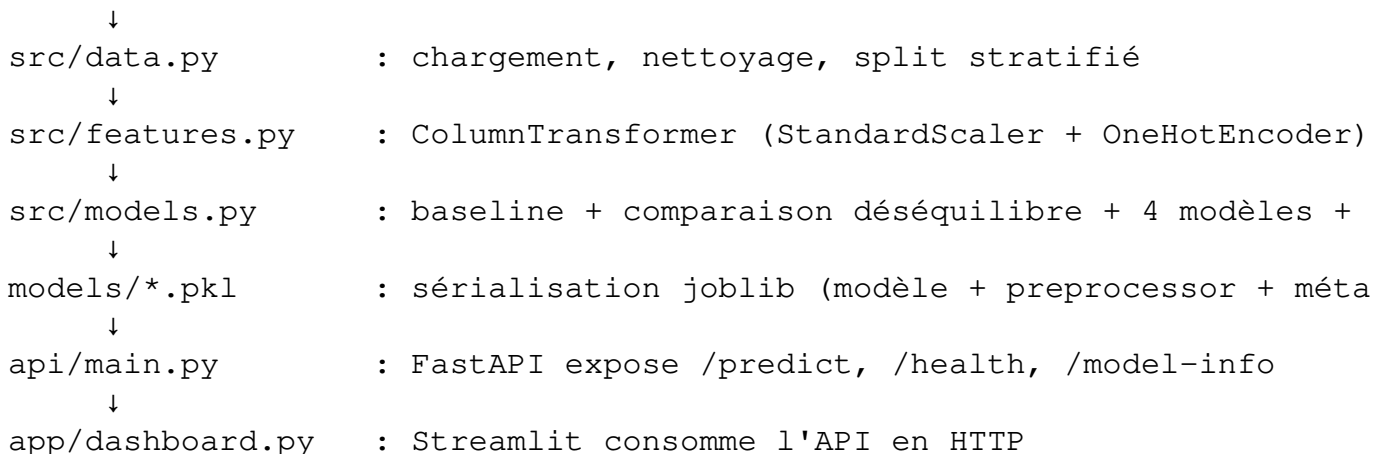
---

## 7. Architecture du pipeline et implémentation

Le projet est structuré comme un mini-produit logiciel et non un notebook monolithique. Le code de production réside dans `src/` (modules `data`, `features`, `models`, `evaluation`,

explain), tandis que les notebooks (01\_eda.ipynb, 02\_modeling.ipynb) servent uniquement à l'exploration et à la traçabilité.

Dataset brut (CSV)



Le pipeline complet est reproductible : `python -m src.models` entraîne, compare, optimise et sérialise en moins de deux minutes sur un CPU standard. La constante `RANDOM_STATE = 42` est centralisée dans `src/config.py` et appliquée partout, garantissant un déterminisme strict. La qualité logicielle est assurée par **90 tests automatisés** (couverture 84 %) dont **7 scénarios critiques** vérifient l'absence de fuite de données, l'application correcte du rééquilibrage, l'idempotence de la sérialisation, la reproductibilité globale, la non-régression de performance, la cohérence API-modèle et le découplage dashboard-modèle.

---

## 8. Mise en œuvre et exploitation de la solution

La solution est exposée à l'utilisateur métier via deux composants opérationnels.

**L'API REST (FastAPI)** sert d'interface programmatique. L'endpoint `POST /predict` accepte un payload JSON validé par Pydantic et retourne la probabilité de churn, la prédiction binaire au seuil 0,35, un niveau de risque qualitatif (faible, modéré, élevé) et le top trois des facteurs explicatifs issus de SHAP. `GET /health` permet aux sondes de monitoring de vérifier que l'API tourne et que le modèle est chargé. `GET /model-info` retourne les métadonnées (date d'entraînement, métriques, seuil, features attendues). La documentation Swagger est servie automatiquement sur `/docs`.

**Le dashboard Streamlit** s'adresse au responsable CRM. Il est volontairement découplé du modèle : il ne charge jamais le `.pkl` directement, il appelle l'API en HTTP. Quatre onglets structurent l'usage : une **vue d'ensemble** avec quatre KPI (clients totaux, clients à risque, revenu global à risque, taux de churn prédit), une **liste priorisée des clients à risque** triable et exportable en CSV, un **simulateur** permettant de saisir un profil client et d'obtenir prédiction et recommandation textuelle, et une vue **facteurs globaux** avec le SHAP summary plot.

Concrètement, une prédiction est utilisée ainsi : un client appelle le support, le conseiller saisit ses caractéristiques dans le simulateur, le dashboard affiche Risque élevé – proba 0,71 avec les trois facteurs principaux (CSAT bas, échecs de paiement répétés, baisse d'activité). Le conseiller peut alors proposer une action ciblée. Au-delà du cas individuel, la vue d'ensemble affiche un revenu à risque global agrégé qui aide la direction financière à anticiper l'impact dans ses prévisions.

---

## 9. Analyse critique, limites et améliorations

Le PR-AUC plafonne à 0,30 et le F1 à 0,41, quelle que soit l'approche d'optimisation essayée. Trois tentatives indépendantes (recherche manuelle, SMOTE forcé, RandomizedSearchCV étendu) convergent vers ce plafond. Cela traduit la **limite de signal du dataset** plutôt qu'un défaut de modélisation : les corrélations Pearson plafonnaient déjà à 0,16 en EDA. Améliorer la performance demanderait des sources de données externes (signaux concurrence, environnement macro, logs d'usage produit fins).

L'absence de **dimension temporelle** est une seconde limite : sans horodatage par observation, le modèle ne capte ni saisonnalité ni dérive. Si l'historique a été collecté sur une période exceptionnelle, le modèle apprend des dépendances temporelles qu'il faudra ré-évaluer en production. Les variables `country` et `city` peuvent par ailleurs introduire un biais géographique si la distribution évolue.

Pour la mise en production, trois améliorations sont recommandées. Premièrement, un **déploiement progressif** par segment (Enterprise d'abord) permet de calibrer le ratio précision / coût d'action sans engager toute la base. Deuxièmement, un **monitoring mensuel de dérive** doit comparer la distribution des prédictions au taux de churn réel observé. Troisièmement, une **boucle de réentraînement** automatisée tous les six mois capturera les nouveaux signaux et les nouvelles modalités de `complaint_type` qui apparaissent.

---

## 10. Conclusion

Le projet livre un système complet allant de la donnée brute à la décision business. La démarche technique a été constante : à chaque étape, un choix justifié par une mesure, et à chaque mesure, une décision actionnable. L'EDA a guidé le preprocessing, le preprocessing a sécurisé le pipeline, la comparaison empirique a sélectionné la stratégie de rééquilibrage, la validation croisée a stabilisé les performances, le tuning du seuil a aligné le modèle sur l'objectif métier, et SHAP a traduit les prédictions en recommandations.

Le Random Forest retenu atteint un Recall de 78,9 % qui dépasse l'objectif business, et son interprétabilité native permet de fournir trois leviers concrets à l'équipe CRM. Au-delà du modèle, le système est industrialisable : modèle sérialisé, API REST documentée, dashboard découplé, 90 tests automatisés, scénarios critiques verts. C'est ce passage d'un score brut à un produit utilisable qui justifie l'effort technique.